

Weird Logic

June 30, 2025

1 Examples

Correctness of Doppler in hyper triple:

$$\{\forall x \in Var, x(\mathcal{P}) = x(L)\} \left[\begin{array}{l} p \in \mathcal{P} : C(p) \\ l \in L : l \end{array} \right] \left\{ \begin{array}{l} \forall l \in L, \exists p \in \mathcal{P} \\ x(p) = x(l) \end{array} \right\}$$

For any program $\mathbf{l}$ written in the language L , there always exists a payload $\mathbf{p}$ such that the program $\mathbf{C}$ and $\mathbf{l}$ execute the same behaviour.

1.1 if-else

Given a Doppler attack language L :

$$L \rightarrow \begin{array}{l} S_1 : x = x + 1 \\ | S_2 : x = x + 2 \end{array}$$

and vulnerable program C :

```
choose a
if a then
  x = x+1
else
  x = x+2
```

write the triple:

$$\{x(\mathcal{P}) = x(L)\} \left[\begin{array}{l} p \in \mathcal{P} \quad , \quad C(p) \\ l \in L \quad , \quad l \end{array} \right] \left\{ \begin{array}{l} \forall l \in L, \exists p \in \mathcal{P} \\ x(p) = x(l) \end{array} \right\}$$

where $P = \{a_\top, a_\perp\}$,

$$\{x(\mathcal{P}) = x(L)\} \left[\begin{array}{c} p \in \{a_\top, a_\perp\} \quad , \quad C(p) \\ l \in S_1 \mid S_2 \quad , \quad l \end{array} \right] \left\{ \begin{array}{c} \forall l \in L, \exists p \in \mathcal{P} \\ x(p) = x(l) \end{array} \right\}$$

Apply GrmDisj-rule:

$$\frac{L \rightarrow L_1 \mid L_2, L \notin L_1, L \notin L_2 \quad \{P\} [C(\mathcal{P}); L_1] \{Q\} \quad \{P\} [C(\mathcal{P}); L_2] \{Q\}}{\{P\} [C(\mathcal{P}); L] \{Q\}}$$

Partition S_1 and S_2 :

$$\forall i \in \{1, 2\}, \{x(\mathcal{P}) = x(L)\} \left[\begin{array}{c} p \in \{a_\top, a_\perp\} \quad , \quad C(p) \\ l \in S_i \quad , \quad l \end{array} \right] \left\{ \begin{array}{c} \forall l \in S_i, \exists p \in \mathcal{P} \\ x(p) = x(l) \end{array} \right\}$$

Apply Weaken-rule:

$$\frac{\mathcal{P} \supseteq \mathcal{P}_1 \quad \{P\} [C(\mathcal{P}_1); L] \{Q\}}{\{P\} [C(\mathcal{P}); L] \{Q\}}$$

Let $\mathcal{P}_1 = \{a_\top\}$; $\mathcal{P}_2 = \{a_\perp\}$, shrink $\mathcal{P}$:

$$\forall i \in \{1, 2\}, \{x(\mathcal{P}) = x(L)\} \left[\begin{array}{c} p \in \mathcal{P}_i \quad , \quad C(p) \\ l \in S_i \quad , \quad l \end{array} \right] \left\{ \begin{array}{c} \forall l \in S_i, \exists p \in \mathcal{P}_i \\ x(p) = x(l) \end{array} \right\}$$

Apply lang-rule:

$$\frac{\exists l, L = \{l\} \quad \{P\} [C(\mathcal{P}); l] \{\exists p \in \mathcal{P}, Q(l, p)\}}{\{P\} [C(\mathcal{P}); L] \{\forall l \in L, \exists p \in \mathcal{P}, Q(l, p)\}}$$

Apply payload-rule:

$$\frac{\exists p \in \mathcal{P}, \{P\} [C(p); l] \{Q(l, p)\}}{\{P\} [C(\mathcal{P}); l] \{\exists p \in \mathcal{P}, Q(l, p)\}}$$

1.2 while-choose

Given a Doppler attack language L :

$$\begin{array}{lcl} L & \rightarrow & S_1 \\ S_1 & \rightarrow & x = x + 1; S_1 \\ & & | \epsilon \end{array}$$

or vulnerable program C :

```
while (choose b) do
  x = x+1
```

write the triple for C :

$$\{\forall l, p, x(p) = x(l)\} \left[\begin{array}{l} p \in \mathcal{P} \quad , \quad C(p) \\ l \in L \quad , \quad l \end{array} \right] \left\{ \begin{array}{l} \forall l \in L, \exists p \in \mathcal{P} \\ x(p) = x(l) \end{array} \right\}$$

where $\mathcal{P} := \mathbb{B}^*$

Apply weaken-rule and shrink $\mathcal{P}$ to $\{b_\top^*; b_\perp\}$

Apply LeftReg-rule:

$$\frac{\begin{array}{c} \mathcal{P} = \{p_\top^*; p_\perp\} \\ \{P \wedge \mathcal{P}_1 = \{p_\top\}\} [C'(\mathcal{P}_1); L_1] \{P\} \end{array}}{\{P\} \left[\begin{array}{l} \text{while } \mathcal{P} \text{ do } C' \\ L \rightarrow (L_1; L) \mid \epsilon \end{array} \right] \{P \wedge \exists p', p = p'; p_\perp\}} \text{LeftReg}$$

Seq-rule (should have more constraints):

$$\frac{\{P\} [C_1(\mathcal{P}_1), L_1] \{H\} \quad \{H\} [C_2(\mathcal{P}_2), L_2] \{Q\}}{\{P\} \left[\begin{array}{l} p \in \mathcal{P}_1 \times \mathcal{P}_2 \quad , \quad C_1(p_1); C_2(p_2) \\ l \in L_1; L_2 \quad , \quad l \end{array} \right] \{Q\}} \text{Seq}$$

SeqLeft-rule:

$$\frac{\begin{array}{c} \{P\} [C_1(\mathcal{P}_1); L \rightarrow (L_1; L) \mid \epsilon] \{H\} \\ \{H\} [C_2(\mathcal{P}_2); L \rightarrow L_2] \{Q\} \end{array}}{\{P\} \left[\begin{array}{l} p \in \mathcal{P}_1 \times \mathcal{P}_2 \quad , \quad C_1(p_1); C_2(p_2) \\ l \in L \rightarrow (L_1; L) \mid L_2 \quad , \quad l \end{array} \right] \{Q\}} \text{SeqLeft}$$

SeqRight-rule:

$$\frac{\begin{array}{c} \{P\} [C_1(\mathcal{P}_1); L \rightarrow L_2] \{H\} \\ \{H\} [C_2(\mathcal{P}_2); L \rightarrow (L_1; L) \mid \epsilon] \{Q\} \end{array}}{\{P\} \left[\begin{array}{c} p \in \mathcal{P}_1 \times \mathcal{P}_2 \\ l \in L \rightarrow (L; L_1) \mid L_2 \end{array}, C_1(p_1); C_2(p_2) \right] \{Q\}} \text{SeqRight}$$

Apply RightReg-rule:

$$\begin{aligned} Q' : \forall l \in L, p \in \mathcal{P}, x(p) = x(l) \wedge \exists p', p = p'; p_{\perp} \\ \implies Q : \forall l \in L, \exists p \in \mathcal{P}, x(p) = x(l) \end{aligned}$$

Get:

$$\{\forall l, p, x(p) = x(l) \wedge \mathcal{P} = \{p_{\top}\}\} \left[\begin{array}{c} p \in \mathcal{P} \\ l \in \{x = x + 1\} \end{array}, \begin{array}{c} x = x + 1 \\ l \end{array} \right] \left\{ \begin{array}{c} \forall l \in L, p \in \mathcal{P} \\ x(p) = x(l) \end{array} \right\}$$

Apply lang-rule and payload-rule.

1.3 do-while

Given a Doppler attack language L :

$$\begin{array}{lcl} L & \rightarrow & S_1 \\ S_1 & \rightarrow & x = x + 1 \\ & & \mid S_1; S_1 \end{array}$$

or language L' :

$$\begin{array}{lcl} L & \rightarrow & x = x + 1; S_1 \\ S_1 & \rightarrow & x = x + 1; S_1 \\ & & \mid \epsilon \end{array}$$

and vulnerable program C :

```
do
  choose a
  x = x+1
while a
```

or vulnerable program C' :

```
b=1
while b>0 do
```

```

b--
choose b
x = x+1

```

or vulnerable program C'' :

```

c=true
while c do
  choose c
  x = x+1

```

TODO: The mapping between program and language is not isomorphic. One solution is to restrict the language to left-regular format when it can be converted. For example, restrict the language to L' as shown above.

Write the triple for C :

$$\{x(\mathcal{P}) = x(L)\} \left[\begin{array}{c} p \in \{\epsilon\} \times \mathcal{P}' \\ l \in \{L \rightarrow x = x + 1; S_1\} \end{array} , \begin{array}{c} C_1(\epsilon); C_2(p') \\ l \end{array} \right] \left\{ \begin{array}{c} \forall l \in L, \exists p \in \mathcal{P} \\ x(p) = x(l) \end{array} \right\}$$

where $\mathcal{P} := \mathbb{B}^*$, $\mathcal{P}' := \{\epsilon\} \times \mathcal{P}'$, and $p' \in \mathcal{P}'$;

$C_1 := x = x + 1$;

$C_2 := \text{while choose } a \text{ do } x = x + 1$

Apply Seq-rule:

$$(1) \{x(\mathcal{P}) = x(L_1)\} \left[\begin{array}{c} p \in \{\epsilon\} \\ l \in \{L_1 \rightarrow x = x + 1\} \end{array} , \begin{array}{c} C_1(p) \\ l \end{array} \right] \left\{ \begin{array}{c} \forall l \in L_1, \exists p \in \mathcal{P} \\ x(p) = x(l) \end{array} \right\}$$

$$(2) \{x(\mathcal{P}') = x(S_1)\} \left[\begin{array}{c} p \in \mathcal{P}' \\ l \in \{S_1 \rightarrow x = x + 1; S_1 | \epsilon\} \end{array} , \begin{array}{c} C_2(\mathcal{P}') \\ l \end{array} \right] \left\{ \begin{array}{c} \forall l \in S_1, \exists p \in \mathcal{P}' \\ x(p) = x(l) \end{array} \right\}$$

(2) is the same as the **while-choose** example.

1.4 choose-while

Given a Doppler attack language L :

$$\begin{array}{lcl} L & \rightarrow & S_1 \\ S_1 & \rightarrow & x = x + 1; S_1 \\ & & | \epsilon \end{array}$$

or L' :

$$\begin{aligned} L &\rightarrow \text{for } i \text{ in } b \text{ do } S_1 \\ S_1 &\rightarrow x = x + 1 \end{aligned}$$

and vulnerable program C :

```
choose b
while b>0 do
  b--
  x = x+1
```

or vulnerable program C' :

```
choose b
while b>0 do
  b = b-2;
  x = x+1
```

Write the triple for C :

$$\{x(\mathcal{P}) = x(L)\} \left[\begin{array}{c} p \in \mathcal{P} \\ l \in L \end{array} , \begin{array}{c} C(p) \\ l \end{array} \right] \left\{ \begin{array}{c} \forall l \in L, \exists p \in \mathcal{P} \\ x(p) = x(l) \end{array} \right\}$$

where $\mathcal{P} := \{b \mid b \in \mathbb{N}\}$.

Borrow the HP rule from unrealisability logic:

$$\frac{\begin{array}{c} \{P\} [C(\mathcal{P}); L] \{Q\} \vdash \{P\} [C(\mathcal{P}); S_1] \{Q\} \\ \dots \\ \{P\} [C(\mathcal{P}); L] \{Q\} \vdash \{P\} [C(\mathcal{P}); S_n] \{Q\} \end{array} \quad L \rightarrow S_1 | \dots | S_n}{\vdash \{P\} [C(\mathcal{P}); L] \{Q\}}_{HP}$$

Get

$$(1) \Gamma \vdash \{x(\mathcal{P}) = x(L)\} \left[\begin{array}{c} p \in \mathcal{P} \\ l \in \{\epsilon\} \end{array} , \begin{array}{c} C(p) \\ l \end{array} \right] \left\{ \begin{array}{c} \forall l \in L, \exists p \in \mathcal{P} \\ x(p) = x(l) \end{array} \right\}$$

$$(2) \Gamma \vdash \{x(\mathcal{P}) = x(L)\} \left[\begin{array}{c} p \in \mathcal{P} \\ l \in \{x = x + 1; S_1\} \end{array} , \begin{array}{c} C(p) \\ l \end{array} \right] \left\{ \begin{array}{c} \forall l \in L, \exists p \in \mathcal{P} \\ x(p) = x(l) \end{array} \right\}$$

where $\Gamma := \{P\} [C(\mathcal{P}); S_1] \{Q\}$

Apply Weaken-rule to (2):

$$\{x(\mathcal{P}) = x(L)\} \left[\begin{array}{c} p \in \{b \mid b \geq 1\} \\ l \in \{x = x + 1; S_1\} \end{array} , \begin{array}{c} C(p) \\ l \end{array} \right] \left\{ \begin{array}{c} \forall l \in L, \exists p \in \mathcal{P} \\ x(p) = x(l) \end{array} \right\}$$

Apply Induction-rule:

$$\frac{xxx}{xxx} \text{Induction}$$

1.5 context-free

Given a Doppler attack language L :

$$\begin{array}{lcl} L & \rightarrow & S_1 \\ S_1 & \rightarrow & x = x + 1; S_1; y = y + 1 \\ & & | \epsilon \end{array}$$

or in the form of $L = \{(x = x + 1)^n; (y = y + 1)^n \mid n \geq 0\}$
and vulnerable program C :

```
choose n
a=n; b=n
while a>0
  a--
  x = x+1
while b>0
  b--
  y = y+1
```

Write the triple for C :

$$\Gamma \vdash \{x(\mathcal{P}) = x(L) \wedge y(\mathcal{P}) = y(L)\} \left[\begin{array}{c} p \in \mathcal{P} \quad , \quad C(p) \\ l \in L \quad , \quad l \end{array} \right] \left\{ \begin{array}{c} \forall l \in L, \exists p \in \mathcal{P} \\ x(p) = x(l) \wedge y(p) = y(l) \end{array} \right\}$$

where $\mathcal{P} := \mathbb{N}$ and $\Gamma := \{a = b\}$.